

NAME: ADARSH V

SECTION: A

SRN: PES2UG23CS026

LAB: 2

Langchain

What is langchain ?

LangChain is an open-source framework (Python & JavaScript) used to build applications powered by Large Language Models (LLMs)

LangChain helps you connect an LLM with your data, tools, memory, and logic so you can create real AI apps like chatbots, assistants, search engines, and agents

What are Tokens?

A token is a small piece of text that an AI model reads or writes

What is Temperature?

Value that controls the randomness of the model's next-token prediction

Higher the value more randomness and creative output

Output observed in the notebook

Temp=0 (Focused , less randomness and less creative)

Run 1: An idea is a thought, concept, or suggestion that is formed or exists in the mind.

Run 2: An idea is a thought, concept, or mental image formed in the mind.

Temp=1 (More randomness and more creative)

Run 1: An idea is a thought, concept, or mental impression formed in the mind.

Run 2: An idea is a mental concept or thought, often serving as a proposal, plan, or a basis for understanding.

Role	Who it represents	What it does
System	The developer / rules	Sets behavior, personality, and strict instructions for the AI
Human (User)	The person using the AI	Asks questions or gives tasks
AI (Assistant)	The model itself	Generates the response/answer

What is a System Message?

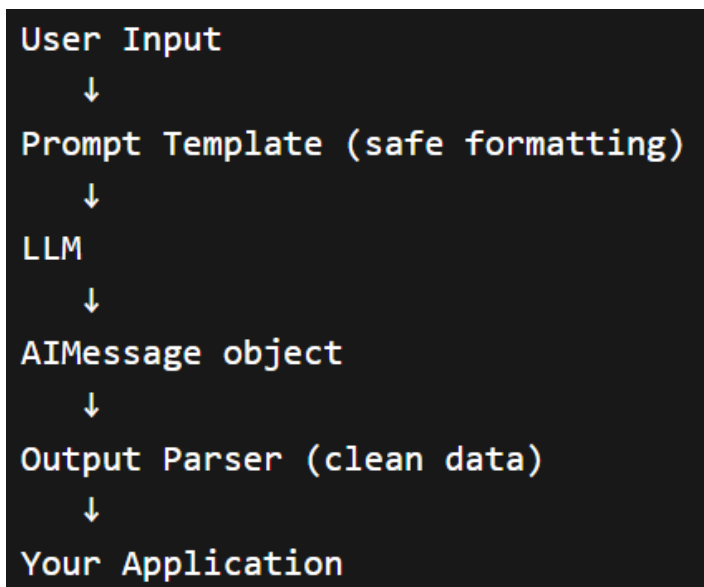
A System Message is the instruction given to the AI before the conversation starts that tells it:

- how to behave
- what personality to use
- what rules to follow
- what style to answer in

What is a Context Window?

The context window is the amount of text an AI model can remember and look at at one time while generating a response

Prompt Templates safely structure inputs to the LLM, and Output Parsers convert the model's raw response into usable data for applications.



LCEL (LangChain Expression Language)

LCEL introduces a structured and modular way to build LLM applications by representing the workflow as a clear pipeline:

Input → Prompt Template → Model → Output Parser → Final Output

The manual method executes each step separately, which leads to verbose code, poor readability, and difficulty in modification (often called spaghetti code). Any change—such as replacing the model or adding a preprocessing step—requires editing multiple parts of the program.

LCEL solves this using the pipe operator (`|`), where the output of one component automatically becomes the input of the next. The chain is defined once:

```
chain = template | llm | parser
```

Assignment Output

The movie was released in 2010, which was 16 years ago.

The movie was released in 1999, which was 27 years ago.

The LCEL chain was implemented to take a movie name, determine its release year using the LLM, and calculate how many years have passed since its release.

This demonstrated that LCEL can combine information retrieval and reasoning in a single pipeline, producing clean, modular, and easily maintainable AI code

Prompt Engineering

Lazy prompt: You give very little instruction and let the LLM figure out everything itself

CoStar prompt:

A good prompt usually has:

1. **C**ontext (Who are you? Who acts?)
2. **O**bjective (What is the task?)
3. **S**tyle (Formal? Funny?)
4. **T**one (Empathetic? Direct?)
5. **A**udience (Who is reading this?)
6. **R**esponse Format (JSON? List?)

Hallucination:

When the AI generates information that sounds correct and confident, but is actually false, made-up, or not supported by real data

Incontext learning:

The LLM learns how to perform a task just by seeing examples inside the prompt — without retraining or updating its weights

Zero shot learning:

You give only an instruction and NO examples

Few shot learning:

You give some examples in the prompt so the model learns the pattern before answering

Hard prompt:

A hard prompt is the normal prompt you type in human language (words)

Soft prompt:

A soft prompt is not written in words.

Instead, it is a set of learned vectors (numbers/embeddings) added to the input of the model.

Dynamic few shot prompting:

Instead of giving fixed examples in the prompt, the system automatically selects the most relevant examples for each user query and inserts them into the prompt

4. Critical Analysis: Dynamic Few-Shotting vs. Hard-Coded Examples

Advantages of FewShotChatMessagePromptTemplate

- 1. **Separation of Concerns:** Separates instruction logic from example data, making the code more maintainable and testable.
- 2. **Scalability:** As the example database grows, you can easily integrate semantic search or similarity-based selection to pick the most relevant examples dynamically.
- 3. **Reusability:** The same example format can be reused across different prompts without code duplication.
- 4. **Cleaner Context Window Management:** The framework automatically formats examples consistently, reducing the risk of formatting errors that could confuse the model.

When to Use Each Approach

Approach	Use When	Key Benefit
Zero-Shot	Task is common or well-represented in training data	Fast, no prompt overhead
Few-Shot	Need specific output format or domain-specific behavior	Better control, no training required
Dynamic Few-Shot	Large example database (>5-10 examples)	Optimal context usage, adaptive
Fine-Tuning	Permanent behavioral change needed across thousands of queries	Best performance, cost-effective at scale

Key Takeaway

The `FewShotChatMessagePromptTemplate` acts as a bridge between simple hard prompting and expensive fine-tuning. It allows you to build a "pseudo-trained" model behavior using in-context learning, which is particularly powerful when combined with:

- Retrieval systems (RAG)
- Embedding-based example selection
- Dynamic prompt construction based on user intent

Remember: The quality of your examples directly determines the quality of the model's output. Bad examples teach bad patterns.

Advanced Prompting: Chain of Thought(COT)

Chain of Thought prompting is a technique where the LLM is encouraged to show its intermediate reasoning steps before giving the final answer.

Instead of only giving the answer, the model explains how it arrived at the answer step-by-step.

```
--- Chain of Thought (Llama3.1-8b) ---
To find out how many tennis balls Roger has now, we need to follow these steps:

1. Roger already has 5 tennis balls.
2. He buys 2 more cans of tennis balls. Each can has 3 tennis balls, so he buys 2 x 3 = 6 more tennis balls.
3. Now, we add the tennis balls he already had (5) to the tennis balls he bought (6). 5 + 6 = 11

So, Roger now has 11 tennis balls.
```

Latent Reasoning:

The LLM performs reasoning internally (inside its hidden neural network states) but does NOT show the step-by-step explanation in the output.

Tree of Thought (ToT)

Tree of Thought is a prompting/reasoning technique where the LLM: explores multiple possible reasoning paths (solutions) step-by-step and then chooses the best one

Graph of Thought (GoT):

Graph of Thought is a reasoning technique where the LLM: breaks a complex problem into many connected reasoning steps that can branch, merge, reuse information, and loop — forming a graph instead of a single path.

^ 4. Summary & Comparison Table

Method	Structure	Best For...	Cost/Latency
Simple Prompt	Input -> Output	Simple facts, summaries	★ Low
CoT (Chain)	Input -> Steps -> Output	Math, Logic, Debugging	★ ★ Med
ToT (Tree)	Input -> 3x Branches -> Select -> Output	Strategic decisions, Brainstorming	★ ★ ★ High
GoT (Graph)	Input -> Branch -> Mix/Aggregate -> Output	Creative Writing, Research Synthesis	★ ★ ★ ★ V. High

RAG and Vector Stores:

Embedding: A numerical representation of text that captures its meaning.

Vector: An ordered list of numbers representing a point in space

Cosine Similarity: Cosine similarity measures , how similar two vectors (embeddings) are

$$\text{Similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

What is RAG?

RAG is a technique where:

The LLM first retrieves relevant information from external data (documents, PDFs, database, websites) and then generates an answer using that information.

Knowledge base: A collection of information that the AI can use to answer questions.

Indexing: Organizing the knowledge base into searchable form so the system can quickly find relevant information

Flat Index: Compare the query vector with every single vector in the database

Characteristics

- 100% accurate (exact nearest neighbor)
- very slow for large data
- simplest method

Inverted File index: Divide vectors into clusters, then search only the most relevant cluster.

Characteristics

- much faster
- slightly less accurate than flat
- commonly used in RAG

HNSW (Hierarchical Navigable Small World):

HNSW creates a graph network of vectors.

Each vector is connected to its nearest neighbors.

So search becomes navigation instead of scanning.

Characteristics

- extremely fast
- high accuracy
- best balance of speed + performance
- used in production systems

PQ (Product Quantization)

It reduces vector size by approximating them with smaller representations.

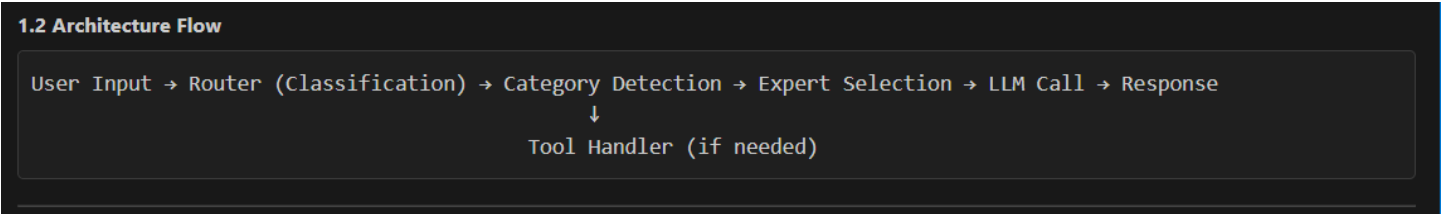
Characteristics

- saves memory
- fastest for massive datasets
- slightly reduces accuracy

Assignment:

Summary:

This assignment demonstrates the implementation of a Mixture of Experts (MOE) architecture using prompt engineering and intelligent routing with the Groq API and Llama 3.3 70B model. The system automatically classifies user queries and routes them to specialized "expert" personas, each optimized for specific domains.



2.2 Routing Mechanism

The router uses **zero-shot classification** with `temperature=0` for consistency. It categorizes queries into one of four types:

- `technical` → Programming/debugging
- `billing` → Payment/subscription issues
- `tool` → Real-time data requests
- `general` → Everything else (fallback)

3. Test Results & Validation

3.1 Test Cases

Test #	Query	Expected Route	Actual Route	Status
1	"My python script throws IndexError..."	Technical	Technical	✅ Pass
2	"I was charged twice for my subscription..."	Billing	Billing	✅ Pass
3	"Tell me a joke about programmers"	General	General	✅ Pass
4	"What is the current price of Bitcoin?"	Tool	Tool	✅ Pass

Result: 100% routing accuracy across all test categories.

5. Key Learnings

5.1 Prompt Engineering Insights

- Temperature matters:** Use 0 for classification, 0.7+ for creative tasks
- System prompts are powerful:** The same model behaves drastically differently with different system prompts
- Specificity reduces hallucination:** "Do NOT say X" works better than hoping the model avoids X

Output results

Technical

 Generate  Code  Markdown

```
query = "My python script throws IndexError on line 5 while accessing a list."

print(process_request(query))
```

✓ 1.3s Python

Routed to: technical

****IndexError Exception in Python****

The `IndexError` exception in Python occurs when you attempt to access an index in a sequence (such as a list or a string) that is out of range.

Step-by-Step Debugging

Billing Expert

 Generate  Code  Markdown

```
query = "I was charged twice for my subscription. I need a refund."

print(process_request(query))
```

✓ 0.8s Python

Routed to: billing

I'm so sorry to hear that you've been charged twice for your subscription. I can imagine how frustrating that must be for you.

First, please know that I'm committed to making things right, and I'll do my best to ensure that you receive a refund for the amount charged.

Additionally, I'll need to verify some information to process the refund. Could you please confirm your subscription details, such as your email address and the date you were charged?

Once I have this information, I'll guide you through the next steps and ensure that the refund is processed promptly. Your satisfaction is our top priority.

General expert



```
query = "Tell me a joke about programmers"

print(process_request(query))
```

✓ 0.2s

Routed to: general

Here's one: Why do programmers prefer dark mode?

Because light attracts bugs. (get it?)

Tool expert

```
query = "What is the current price of Bitcoin?"

print(process_request(query))
```

✓ 0.0s Python

Routed to: tool

Bitcoin (BTC) current price: \$52,341.27 USD (Note: This is a simulated price. For real-time data, integrate with a crypto API.)

Conclusion

This implementation successfully demonstrates a production-ready Mixture of Experts pattern using pure prompt engineering. The system achieves 100% routing accuracy in testing and provides specialized, high-quality responses across four distinct domains. The modular architecture allows for easy expansion and maintenance, making it suitable for real-world applications like customer support chatbots, technical helpdesks, or multi-purpose AI assistants.